



Building

Given a customer scenario, explain how source control branching/versioning/merging can be used and recommend appropriate strategies.

Develop your career with Salesforce Training and Certification Preparation

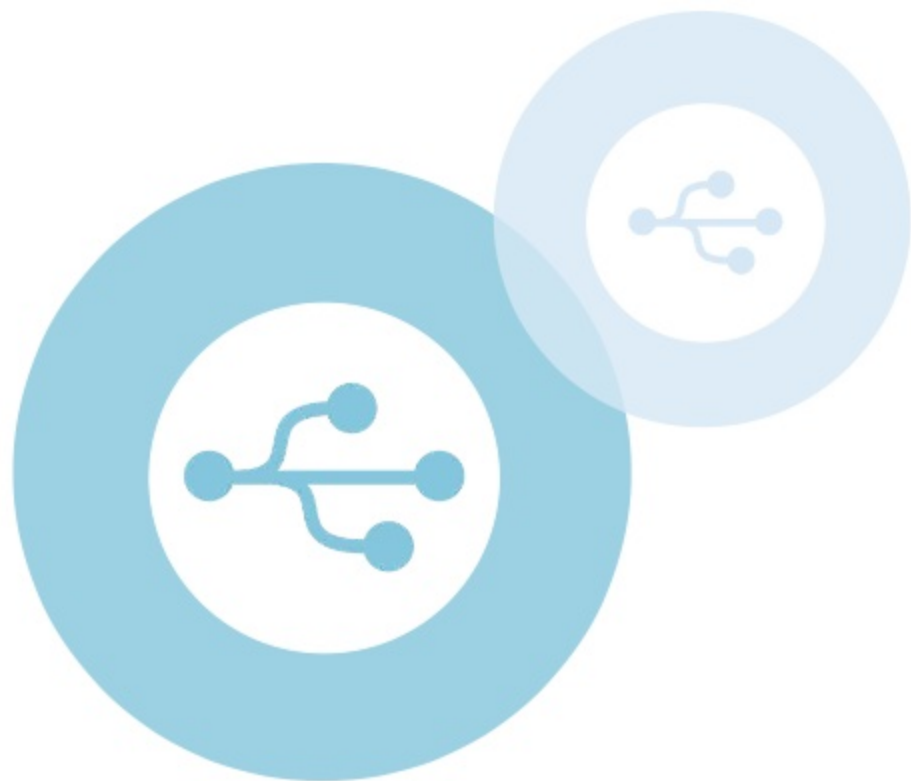
Table of Contents

-  [Source Control](#)
-  [Branching Strategies](#)
-  [Commits & Merging](#)
-  [Business Scenario](#)
-  [Requirements & Solutions](#)



After studying this topic, you should be able to:

-  Describe the various features of a source control system.
-  Explain various branching strategies that can be used for a source control workflow.
-  Identify the considerations related to the use of commits and merges in Git.
-  Recommend appropriate source control strategies for various business requirements.

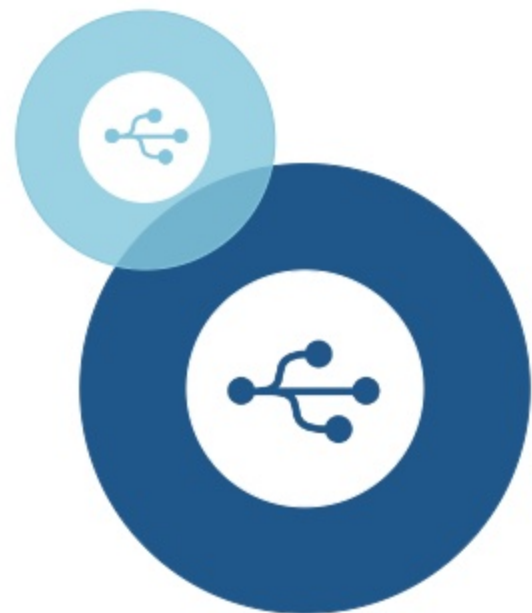


Introduction

Using a **source control system** like Git for Salesforce development allows tracking and managing changes to code and configuration over time. It allows developers to revert to previous versions, compare changes, and merge updates from multiple sources.

Git keeps track of changes in a project over time while **GitHub** is a collaboration platform that hosts Git repositories. There are various **branching strategies** available for Git workflows, including **Centralized Workflow**, **GitHub Flow**, **Gitflow Workflow**, **Forking Workflow**, and **Trunk-Based Development**.

A **commit** represents a snapshot of a project at a specific point in time. A **merge conflict** should be resolved to merge two branches successfully. It occurs when two developers have changed the same lines in a file or if a developer deleted a file while another developer was modifying it.



Source Control System

Branching Strategies

Various branching strategies are available for version control branching. These include Centralized Workflow, GitHub Flow, Gitflow Workflow, Forking Workflow, and Trunk-Based Development.

Commits

A commit represents a snapshot of a project at a specific point in time. Semantic Versioning can be used in commit messages.

Merging

A merge refers to the combined history of two or more branches. Git uses two merging strategies called 'recursive merge' and 'fast-forward merge'.

Source Control

A source control system, such as Git, can be used to track and manage changes to code and configuration over time.

Git & GitHub

Git is a version control application that keeps track of everything related to project changes over time. GitHub is a collaboration platform that hosts Git repositories.



Source Control

[Table of Contents](#) 

Source Control

A source control system **tracks and manages changes to code and configuration** over time. It allows developers to **revert** to previous versions, **compare** changes, and **merge** updates from multiple sources.

❖ COLLABORATION

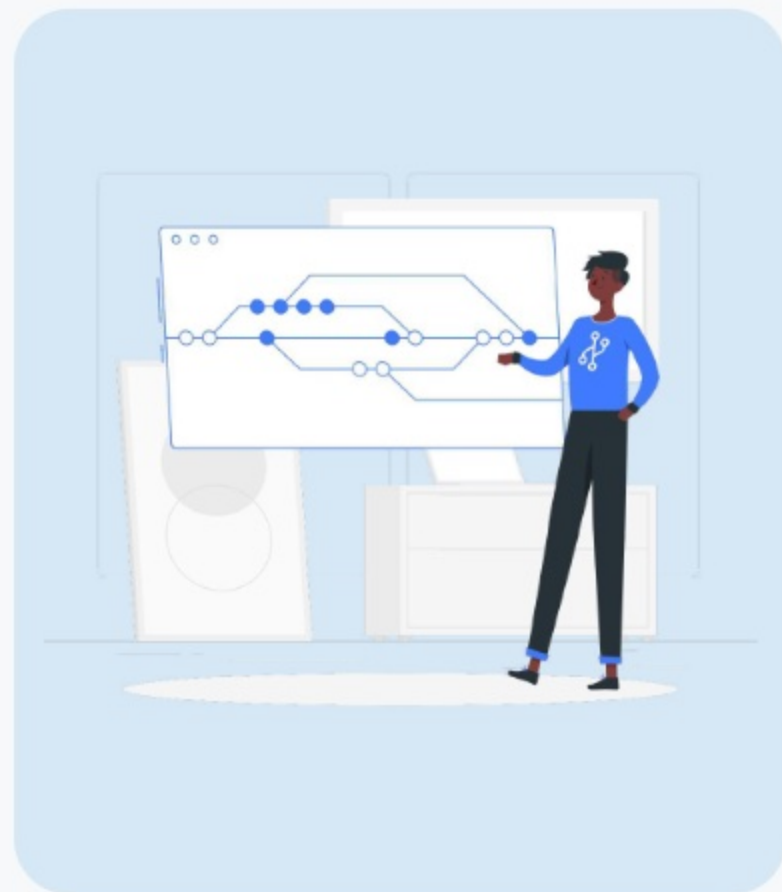
A source control system, such as **Git**, facilitates **collaboration** among developers by allowing them to **work on the same project without overwriting each other's changes**.

❖ VERSION TRACKING

Versioning tracking allows **auditing changes**, understanding **project evolution**, and identifying **when and why specific changes were made**.

❖ IMPROVED PROCESS

Code and metadata can be moved between Salesforce environments with **accuracy and efficiency**, enhancing the development process.



Source Control

A source control system **tracks and manages changes to code and configuration** over time. It allows developers to **revert** to previous versions, **compare** changes, and **merge** updates from multiple sources.

❖ BACKUP & ROLLBACK

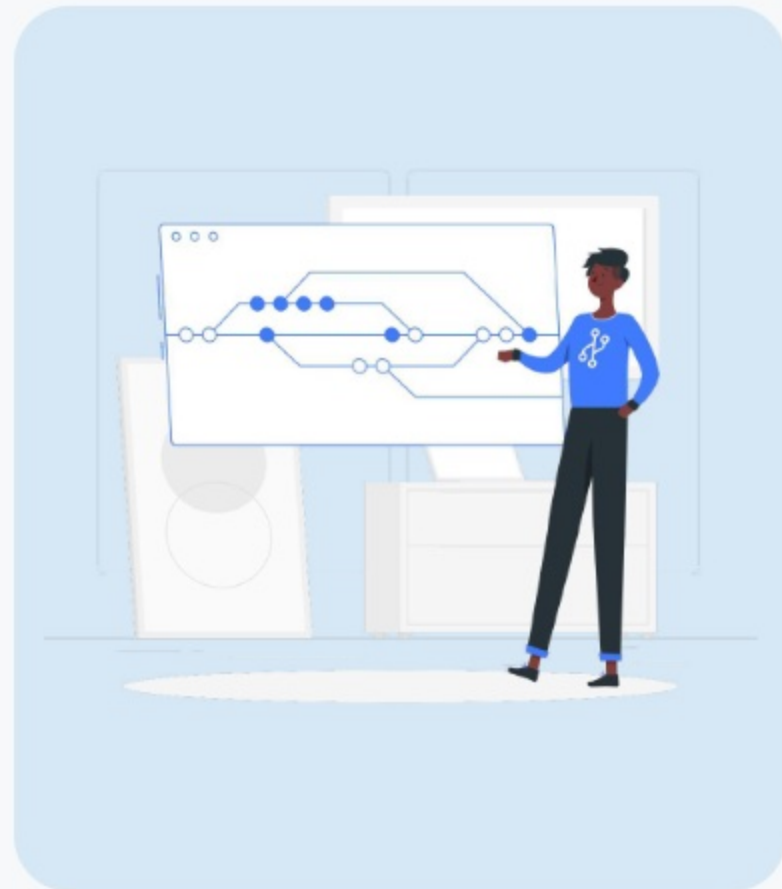
A source control system provides **backup and rollback capabilities**. Code and configuration changes are stored. A developer can **revert to earlier versions** of any file if a new deployment causes issues.

❖ ISOLATED DEVELOPMENT

Distributed development teams working in different regions on different projects can **work in isolation**.

❖ MULTI-SANDBOX STRATEGY

In a multi-sandbox strategy, **code reviews** can be performed before deployment, and each **feature or change can be tracked separately**.



Salesforce DX and Source-Driven Development

Salesforce DX (Developer Experience) is a **suite of development tools** for the Lightning platform that enables **source-driven development**.

❖ SOURCE OF TRUTH

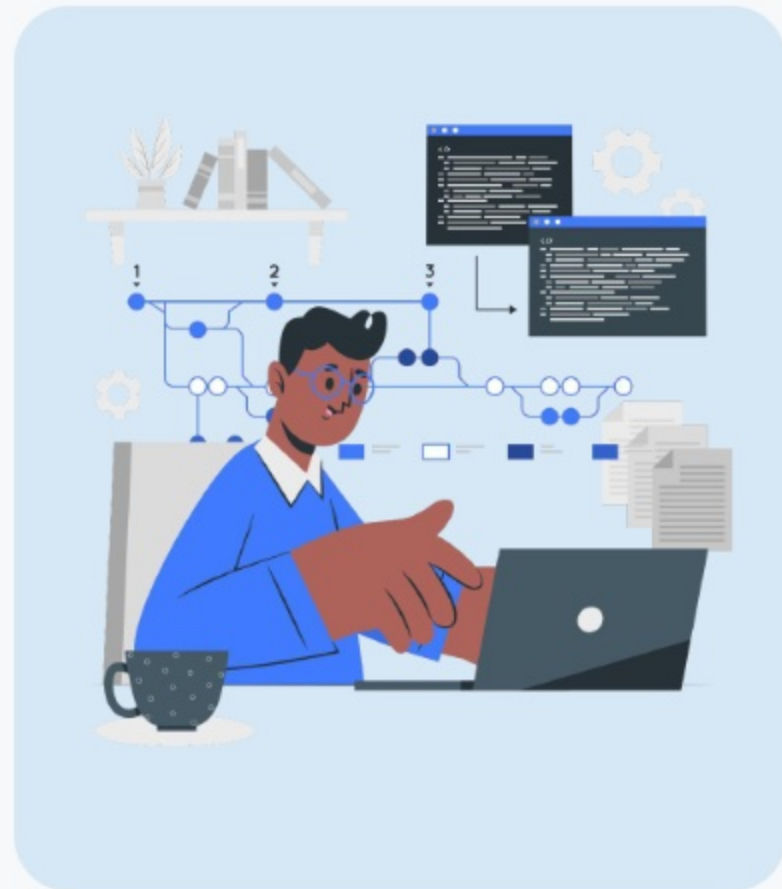
The **version control system** is the source of truth for the **source code and metadata**, providing more agility to develop Salesforce apps.

❖ SALESFORCE CLI

Salesforce CLI can be used to **manage code and metadata** across the entire development lifecycle.

❖ SCRATCH ORGS

Flexible and configurable scratch orgs can be used for development and testing, making it easier to build and deploy applications.



Salesforce DX and Source Tracking

Source Tracking can be **enabled in a production org** to automatically track changes between **sandboxes** created from it and **Salesforce DX projects**.

❖ SOURCE TRACKING

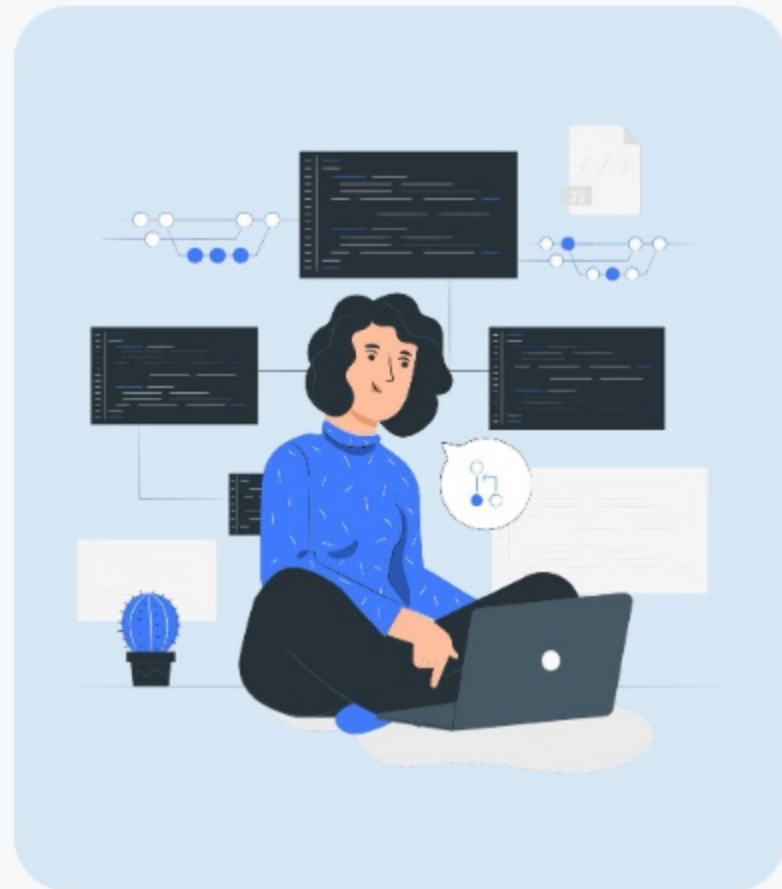
Source tracking can be used to **automatically track changes** to metadata components **between the local project and a scratch org or sandbox**. It only affects the local project, not the org.

❖ SUPPORTED ORGS

Developer and **Developer Pro sandboxes** are supported for source tracking if the associated production org is enabled for source tracking.

❖ OTHER CAPABILITIES

Changes deployed to a sandbox by other developers can be viewed.
Conflicts can be identified and resolved.



Git

Git is a **version control application** that keeps track of everything related to **project changes** over time. There are various key concepts and terms in Git.

❖ REPOSITORIES

A repository is a **collection of source files** used to compile the project.

❖ COMMITS

A commit is a **snapshot of a project** at a specific point in time. Commits are created when **discrete units of work** are added or removed.

❖ BRANCH

A branch is a **series of commits** that represent changes in the project over time. Each repository has a **default branch** (called '**main**') that contains **production-ready version** of the code.



Git

Git is a **version control application** that keeps track of everything related to **project changes** over time. There are various key concepts and terms in Git.

❖ ADDITIONAL BRANCHES

Additional branches can be created for **new features, fixing bugs, or making other project changes**.

❖ MERGE

A merge refers to the **combined history of two or more branches**. The **feature branch** is typically merged with the **default or deployed branch** of the repository to move the features into production.

❖ TAG

A tag is a **pointer to a specific commit** that provides a persistent reference to an event. Tags are typically used with **semantic versioning**.



GitHub

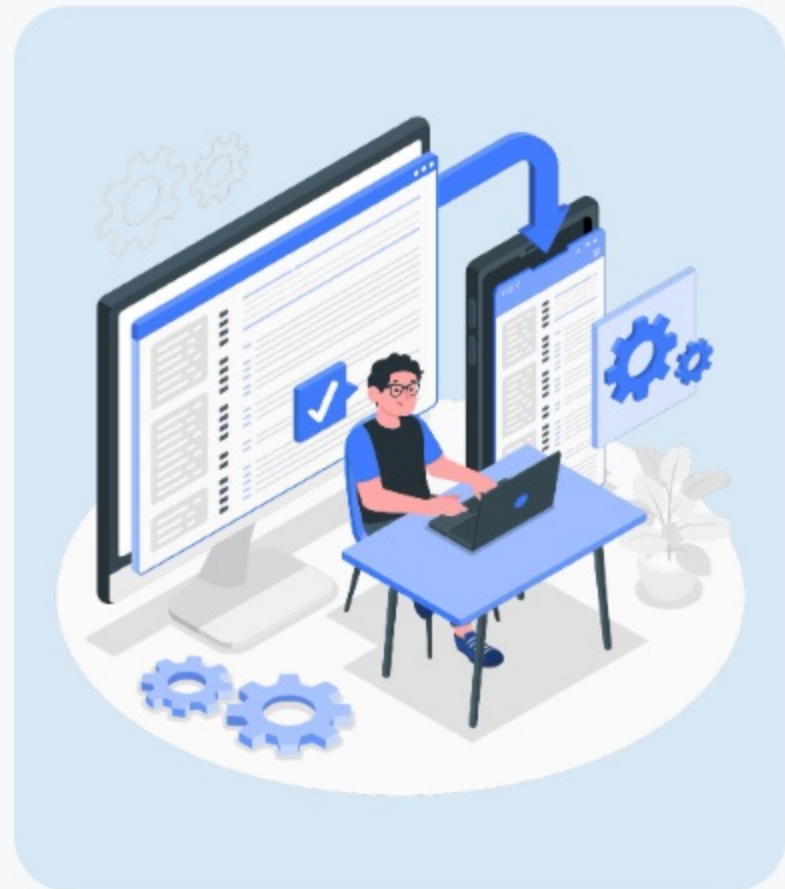
GitHub is a **collaboration platform** that hosts **Git repositories** and acts as a **remote source of truth** for the shared work of developers.

❖ ISSUES

Issues are **threaded discussions** about the project. They can be used to **track bugs and feature requests** and **assigned to specific team members**.

❖ PULL REQUESTS

A pull request is a **change to the code** on a branch that the author would like to **incorporate in another branch**. It provides a place to **discuss the proposed changes** and **invite other team members** to **comment and complete code reviews**. It also shows the **result of automated tests**.



Branching Strategies

Table of Contents 

Branching Strategy

When choosing a **suitable branching strategy** for a GitHub workflow, various considerations apply.

❖ SHORT-LIVED BRANCHES

Generally, branches should be short-lived and **created to accomplish a specific function**. They should be **deleted** once they are **merged** to prevent confusion, encourage up-to-date code, and facilitate iterative improvements to projects.

❖ LONG-LIVED BRANCHES

A long-lived branch is suitable for a **development branch** or a situation that requires **multiple levels of deployment-level code**. However, there are challenges with a long-running branch, such as **not having the most up-to-date version of each branch** and **needless merge conflicts**.



Branching Strategy

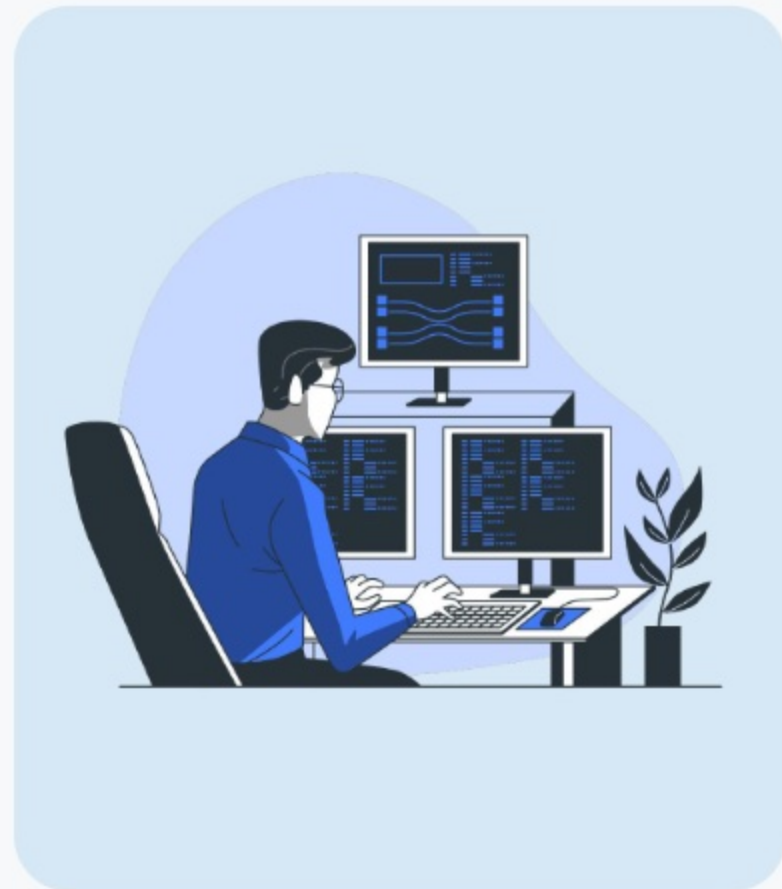
When choosing a **suitable branching strategy** for a GitHub workflow, various considerations apply.

❖ BRANCHING ELEMENTS

Some important aspects to consider for creating branches include **naming conventions**, **labels**, **assignees**, **milestones**, **project boards**, **templates** for issues and pull requests, **sign-off** process, **users** responsible for **merging pull requests**, etc.

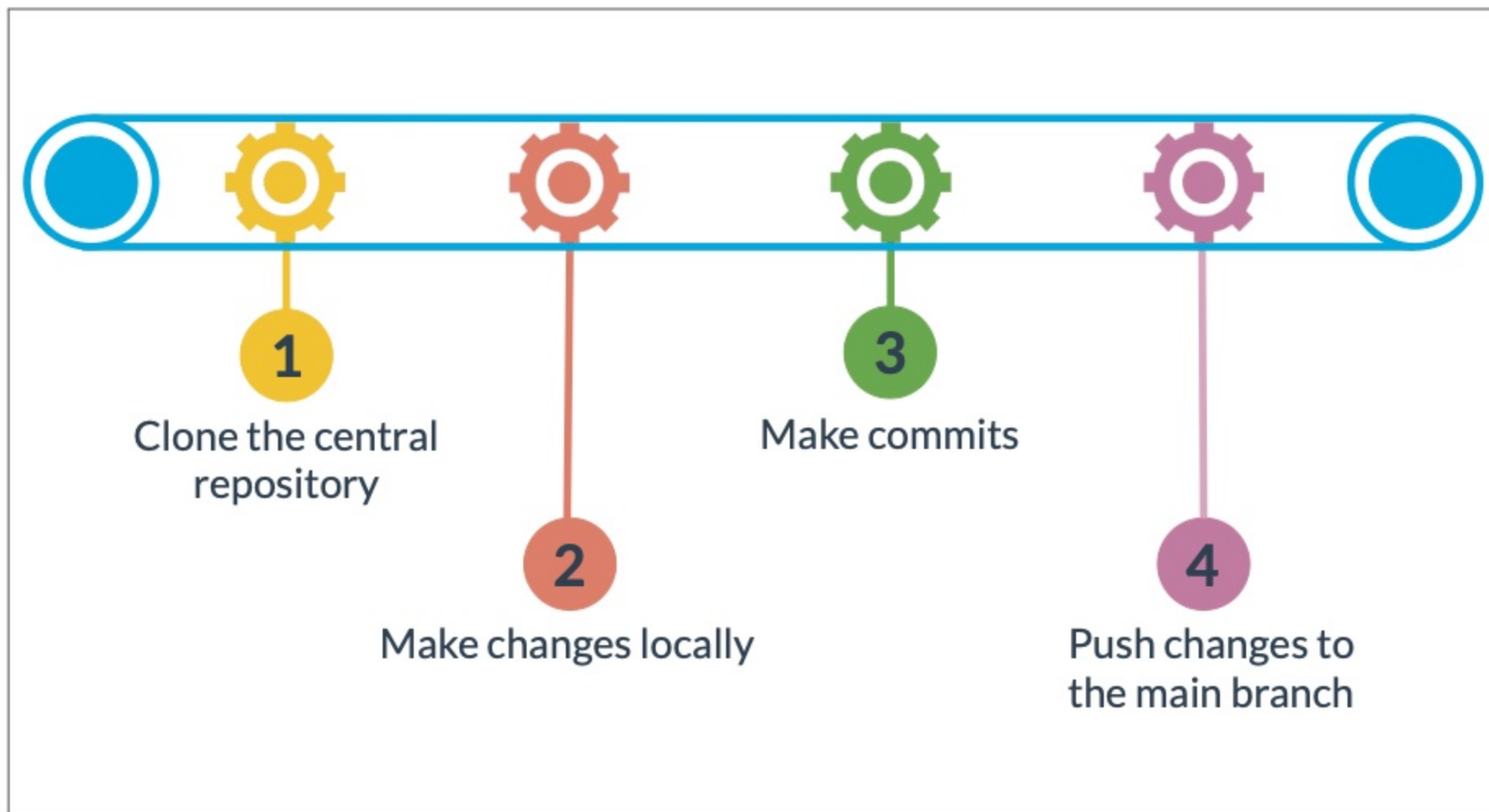
❖ FACTORS

Some important factors to consider for choosing a suitable branching policy are **team size**, **team structure** and **complexity**, **desired release cadence**, and **existing workflow**.



Centralized Workflow

The Centralized Workflow is a **GitHub workflow** that uses a **single central repository**.



Centralized Workflow

The Centralized Workflow is a **GitHub workflow** that uses a **single central repository**.

Clone the central repository

A developer clones the central repository, creating a local copy of the entire project.

Make changes locally

The developer makes changes to the local repository by adding, editing, and deleting files.

Make commits

Changes are staged and committed to track modifications in the local project.

Push changes to the main branch

After finalizing and committing the local changes, they are pushed to the main branch of the central repository.

Centralized Workflow

The Centralized Workflow is **straightforward and easy to understand**, making it suitable for small teams looking for simplicity.



SMALL TEAMS

It is best for **small teams or individual developers** when tightly coordinated development efforts are required, minimizing the risk of conflicting changes.



SIMPLE COLLABORATION

It is ideal when a project has **simple collaboration needs** and does not require a complex branching strategy for features or releases.

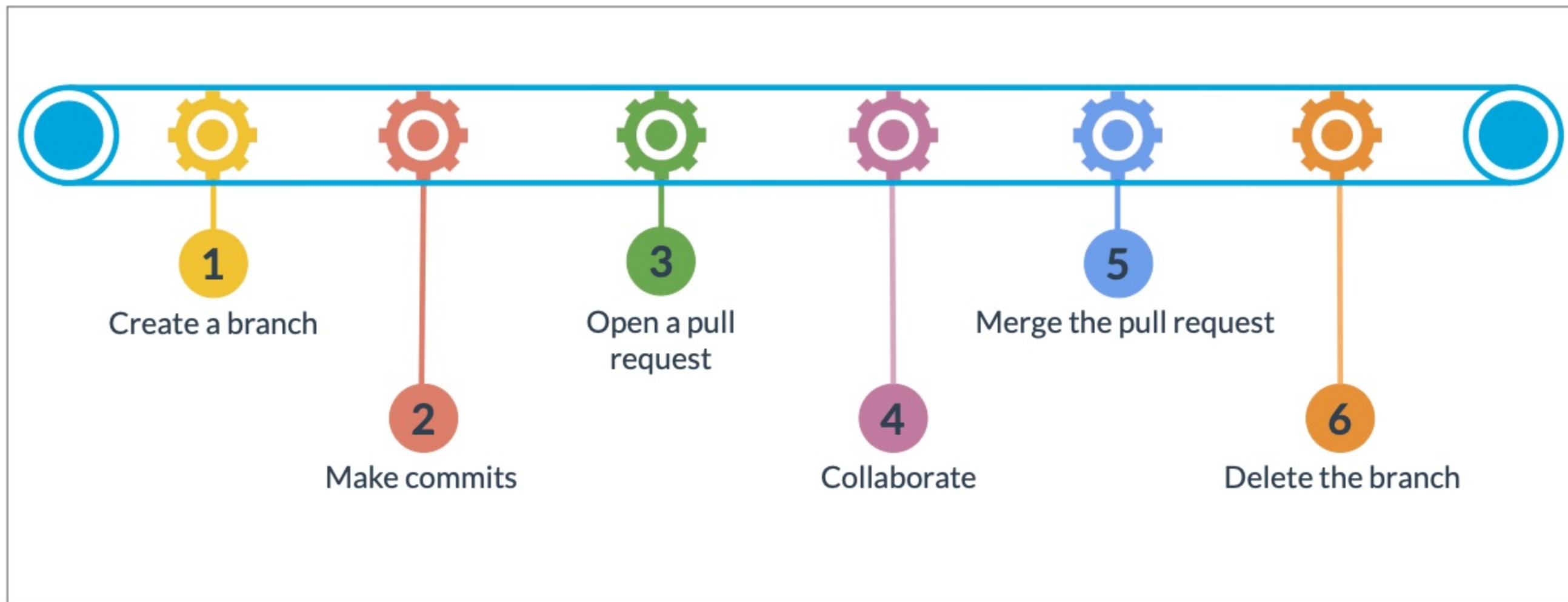


TRANSITION

It is suitable for teams that are **moving from a centralized version control system to Git**, offering a familiar structure and easing the transition process.

GitHub Flow

The GitHub flow is a **lightweight, branch-based workflow** used to collaborate on projects.



GitHub Flow

The GitHub flow is a **lightweight, branch-based workflow** used to collaborate on projects.

Create a branch

To work on a new feature or fix a bug, a new branch is created from the main branch. Any changes made are only reflected in the branch.

Make commits

Changes made to files on the new branch are committed using descriptive message. After the commit, the changes are pushed to the branch.

Create a pull request

A pull request is created to discuss the proposed changes. It can include a summary of the changes made and what problem they solve.

Collaborate

Reviewers can leave comments, questions, and suggestions on the proposed changes. Further changes can be committed and pushed in response.

Merge the pull Request

Once the pull request is approved, the changes are deployed to the target environment, and the pull request is merged with the main branch.

Delete the branch

After the pull request is merged, the branch is deleted, which indicates that the work is complete and prevents users from accidentally using it.

GitHub Flow

The GitHub Flow is typically used in projects that prioritize a **simple, clear workflow** with a **continuous integration/deployment** model.



RAPID ITERATIONS

It is ideal for projects that aim for **continuous integration/continuous delivery (CI/CD)**, where changes need to be deployed quickly and efficiently after passing automated tests.



HIGH COLLABORATION

It is suitable for teams that value **collaboration and fast feedback loops** with stakeholders. It facilitates **easy code reviews and discussions** via pull requests.

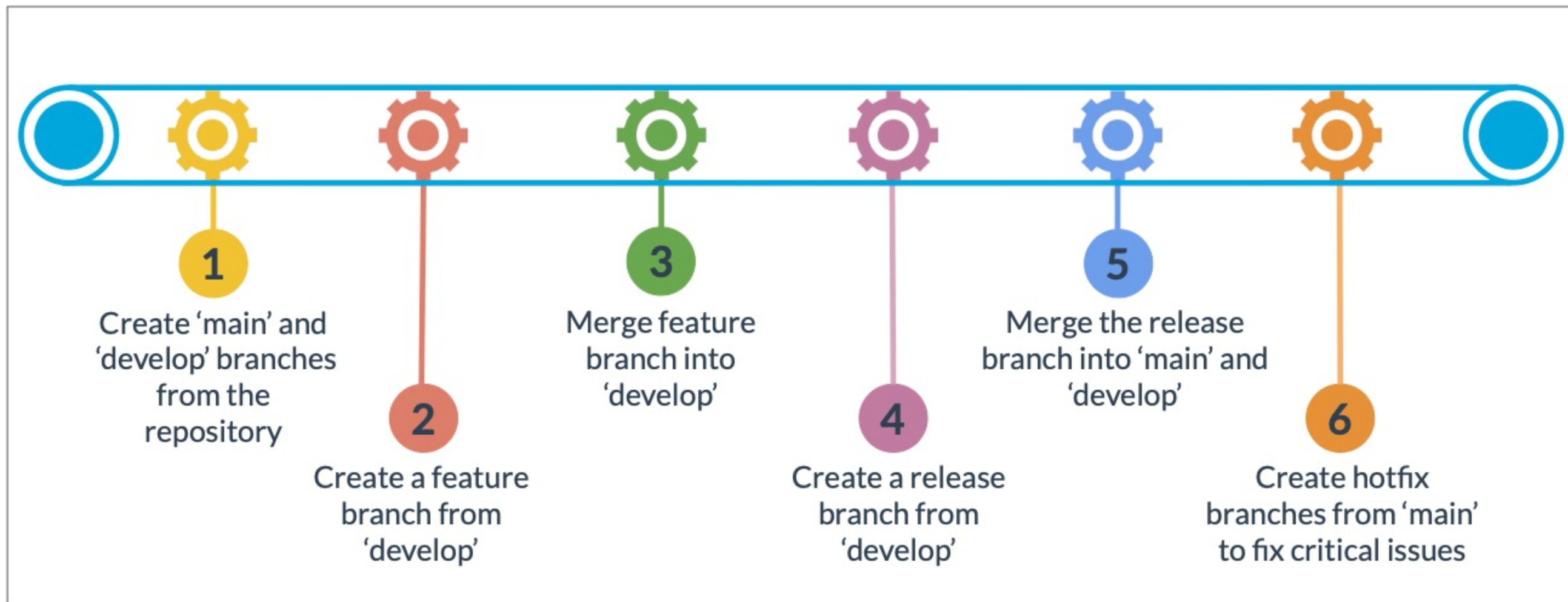


SIMPLIFIED WORKFLOW

It is suitable when a **streamlined workflow** is required **without the overhead** of managing multiple branches or complex merging strategies.

Gitflow Workflow

The Gitflow Workflow is a branching model that uses **two primary branches** and **features branches**.



Gitflow Workflow

The Gitflow Workflow is a branching model that uses **two primary branches** and **features branches**.

Create primary branches

The repository is cloned and two primary branches called 'main' and 'develop' are established.

Create a feature branch

A feature branch is created from the 'develop' branch to work on a new feature.

Merge with the develop branch

After feature completion, the feature branch is merged into the 'develop' branch.

Create a release branch

A release branch is created from 'develop' when ready for a new release.

Merge with develop and main

When the release is complete, the release branch is merged with 'develop' and 'main'.

Create hotfix branches

Hotfix branches are created from 'main' to work on urgent fixes. They are merged back into 'main' and 'develop' after completion.

Gitflow Workflow

The Gitflow Workflow is typically used to manage **larger projects** for **parallel development of features**, **preparation for upcoming releases**, and **quick fixes**.



SCHEDULED RELEASES

It is ideal for projects with **defined release schedules** that require a structured approach for managing development, releases, and hotfixes.



COMPLEX PROJECTS

It is suitable for **complex projects** that require maintaining **multiple versions** in production, staging, and development environments.

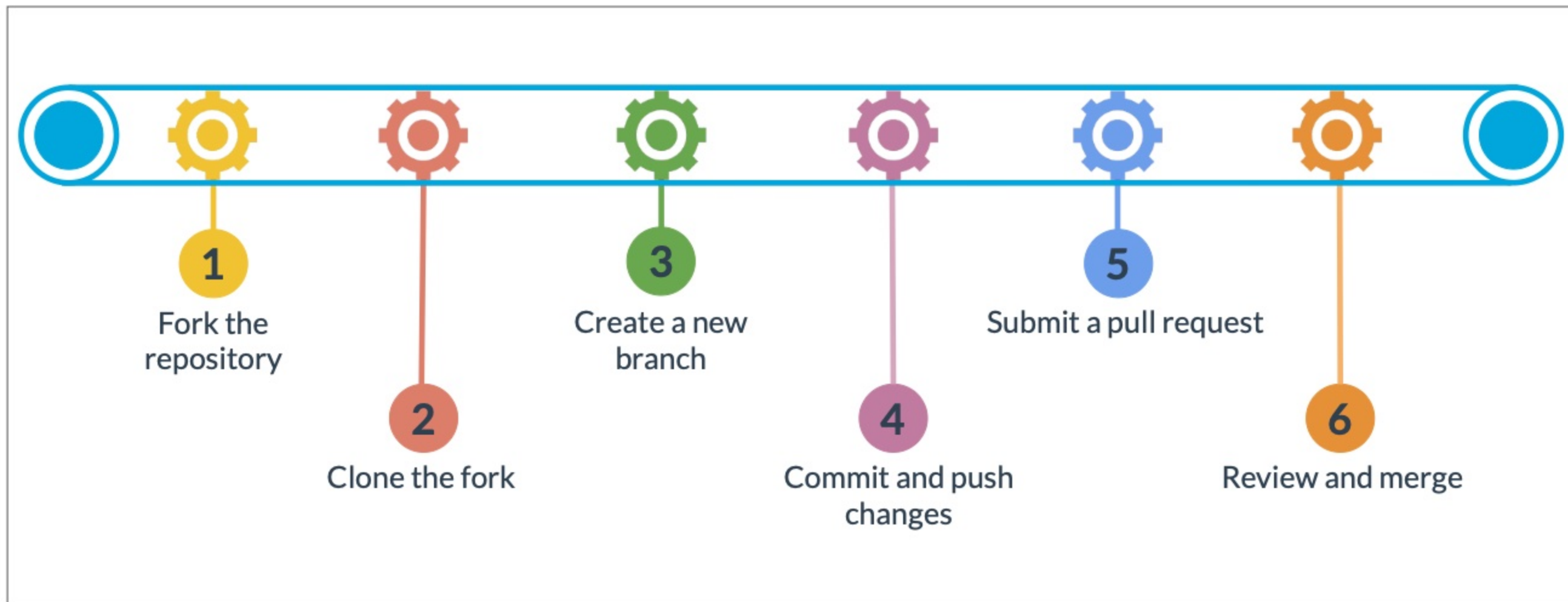


COLLABORATION

It provides an **organized, standard methodology** that **clearly defined roles and responsibilities** for managing large-scale projects.

Forking Workflow

In the Forking Workflow, a developer has a **private local repository** and a **public server-side repository**.



Forking Workflow

In the Forking Workflow, a developer has a **private local repository** and a **public server-side repository**.

Fork the repository

A personal copy of the repository is created on the server side.

Clone the fork

The fork is cloned to the local machine for development work.

Create a new branch

In the local repository, a new branch is created to work on new features or bug fixes.

Commit and push changes

Changes are committed regularly to the new branch and pushed to the fork.

Submit a pull request

A pull request is created from the fork to the original repository to propose the changes for integration.

Review and merge

The pull request is reviewed. Once approved, the changes are merged.

Forking Workflow

The Forking Workflow is suitable for **open-source projects** or scenarios that require contributors to **work independently**.



OPEN SOURCE

It is ideal for **open-source projects** where contributors are **not directly affiliated with the project** or **do not have write access** to the main repository.



CONTROL

When external contributions need to be **reviewed and vetted** thoroughly, project maintainers can **control what gets merged and when**.

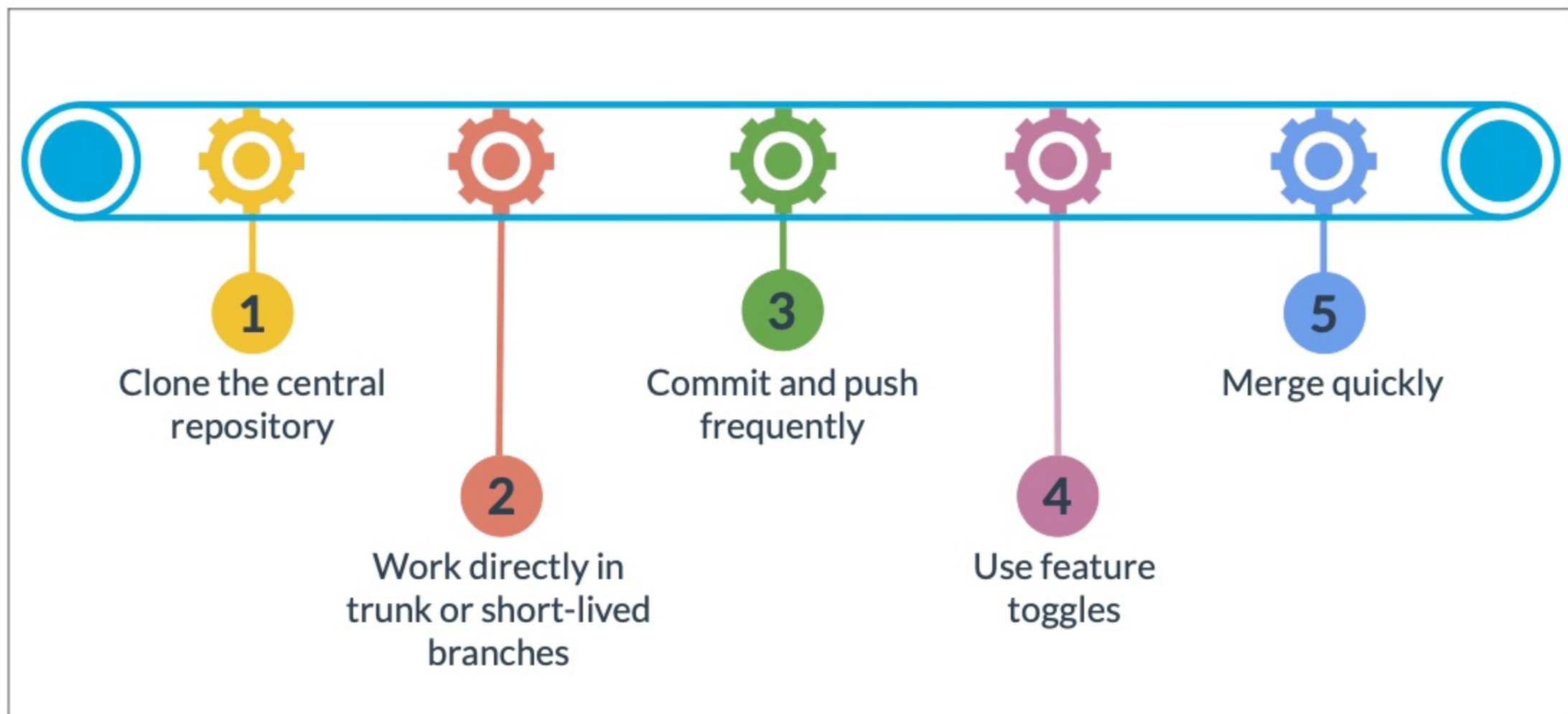


LARGE TEAMS

It is suitable for **large teams that are distributed geographically**, allowing each contributor to work independently.

Trunk-Based Development (TBD)

In Trunk-Based Development (TBD), **small, frequent updates** are merged into the **trunk** or **main branch**.



Trunk-Based Development (TBD)

In Trunk-Based Development (TBD), **small, frequent updates** are merged into the **trunk** or **main branch**.

Clone the central repository

The central repository is cloned to start with a fresh clone of the main codebase.

Work directly in trunk

Work is performed directly in the core trunk or main branch. Alternatively, it is performed in a short-lived branch.

Commit and push frequently

Small, incremental changes are made and pushed at least daily to minimize divergence from the trunk.

Use feature toggles

Incomplete features can be hidden from users without removing them from the 'main' branch.

Merge quickly

If using a short-lived branch, it is merged quickly into the 'main' branch after a quick review process.

Trunk-Based Development (TBD)

Trunk-Based Development (TBD) is suitable for teams that **prioritize rapid delivery** and **wish to avoid the complexity of long-lived branches**.



RAPID RELEASE CYCLES

It is suitable for teams that are committed to **CI/CD practices** and want to focus on **rapid release cycles**, minimizing branch management overhead and integration challenges.



AGILE & LEAN PRACTICES

It is ideal when teams want to focus on **agile development practices** to build, test, and release software in short, iterative cycles, promoting a faster feedback loop.



MINIMIZING CONFLICTS

Frequent commits to the **main branch** are encouraged, **reducing the scope and complexity of merge conflicts** by avoiding long-lived branches.

Commits & Merging

[Table of Contents](#)



Commits

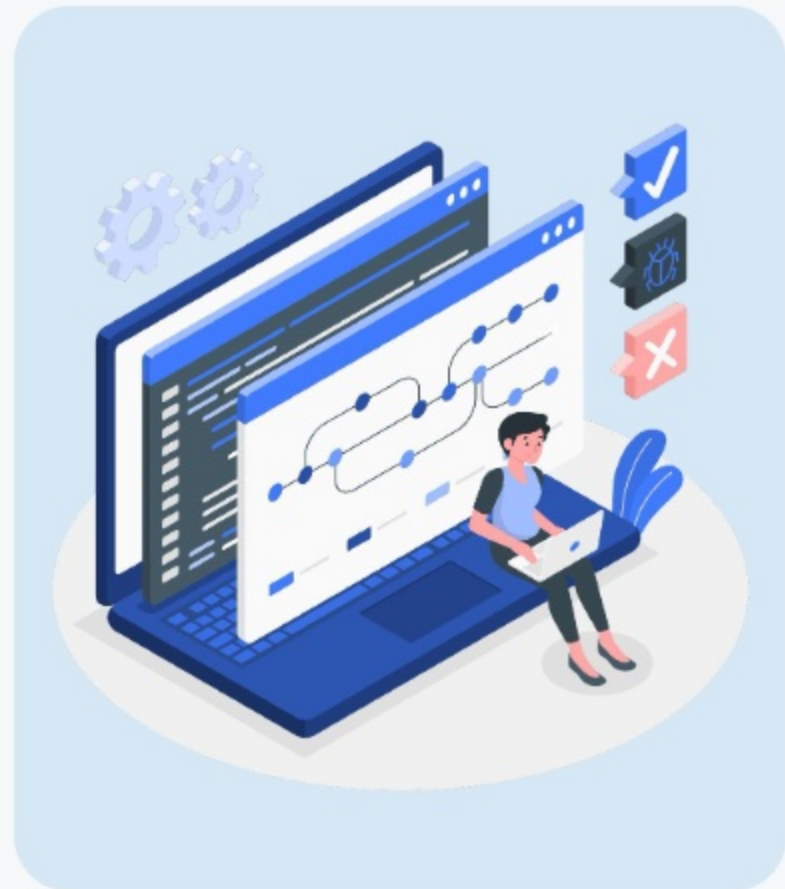
A commit represents a **snapshot of the project** that contains a lot of information.

❖ SHA-1 HASH

Each file in a snapshot is given a **unique SHA-1 hash** called a **blob**. The blobs are referenced by a **tree**, which is referenced by the commit. As changes are made to a file and new commits are made, a **new SHA-1 hash** is applied to the file, which **references the previous hash** as the parent.

❖ COMMIT HISTORY

Each **new commit** references the **previous commit as the parent**. This **parent-child relationship** creates a consistent history and allows Git to **merge branches together**.



Commits

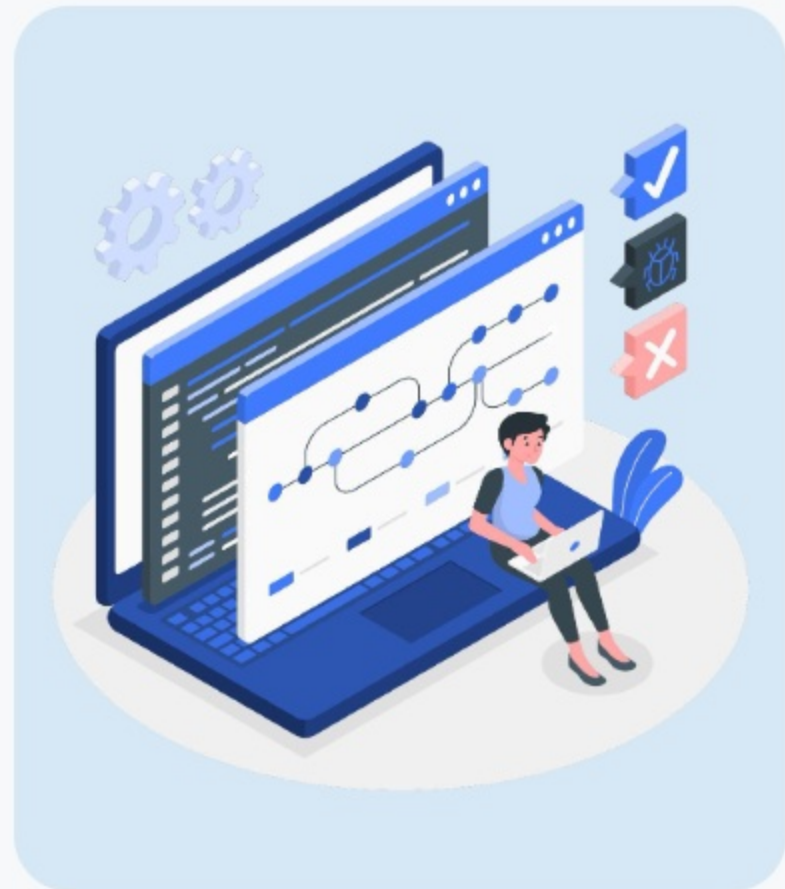
A commit represents a **snapshot of the project** that contains a lot of information.

❖ SEMANTIC VERSIONING

Semantic Versioning can be used in commit messages to **track the evolution of code and configurations** with **clear version numbers** that indicate the nature of changes (major, minor, or patch).

❖ RELEASE TAGS

Tags can be used in Git to **mark releases or significant changes**, making it easier to deploy specific versions to Salesforce orgs or rollback if necessary.



Commits

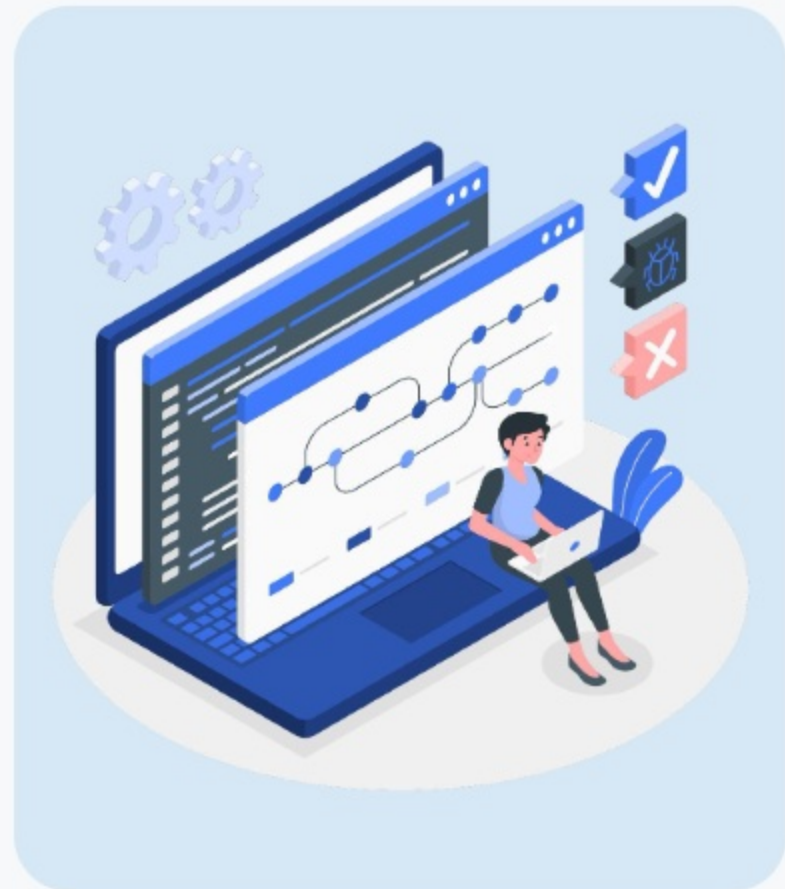
A commit represents a **snapshot of the project** that contains a lot of information.

❖ COMMIT HISTORY

Git can be used to display a **list of all the commits on the current branch**. The project history can also be accessed on GitHub.

❖ COMPARISONS

When working on a commit, **versions of files can be compared**. The differences between what is currently in the **working directory** and the **staging area** can be help add the right files to the commit.



Commits

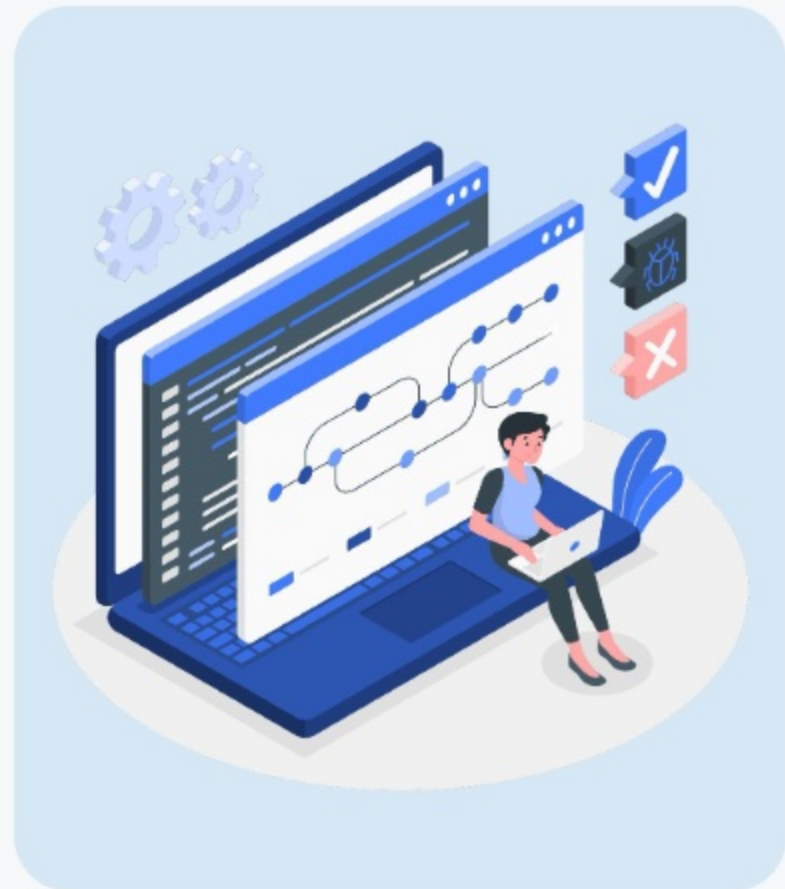
A commit represents a **snapshot of the project** that contains a lot of information.

❖ UNDOING CHANGES

Git can be used to undo a change by **creating a new commit** with changes that are the opposite of the changes in the original commit. It also allows **making a modification to the last commit**, which alters the commit history of the project.

❖ REWINDING

Git can be used to **rewind the history** of the project, which also alters the commit history. It is only recommended when **commits have not been pushed** to the remote branch.



Merge Strategies

Git uses two primary methods for merging changes: **Recursive Merge** and **Fast-Forward Merge**

❖ RECURSIVE MERGE

When the feature branch **doesn't have the latest version of code in the target branch (usually main)**, recursive merge is used. When a pull request is created and changes are merged, a **merge commit** is created, which combines the changes in the two branches.

❖ FAST-FORWARD MERGE

When there are **no new commits** other than the ones in the feature branch, fast-forward merge is used. In this case, Git **does not create a new merge commit**. The tip of the target branch is fast-forwarded to include the changes in the feature branch.



Merge Conflicts

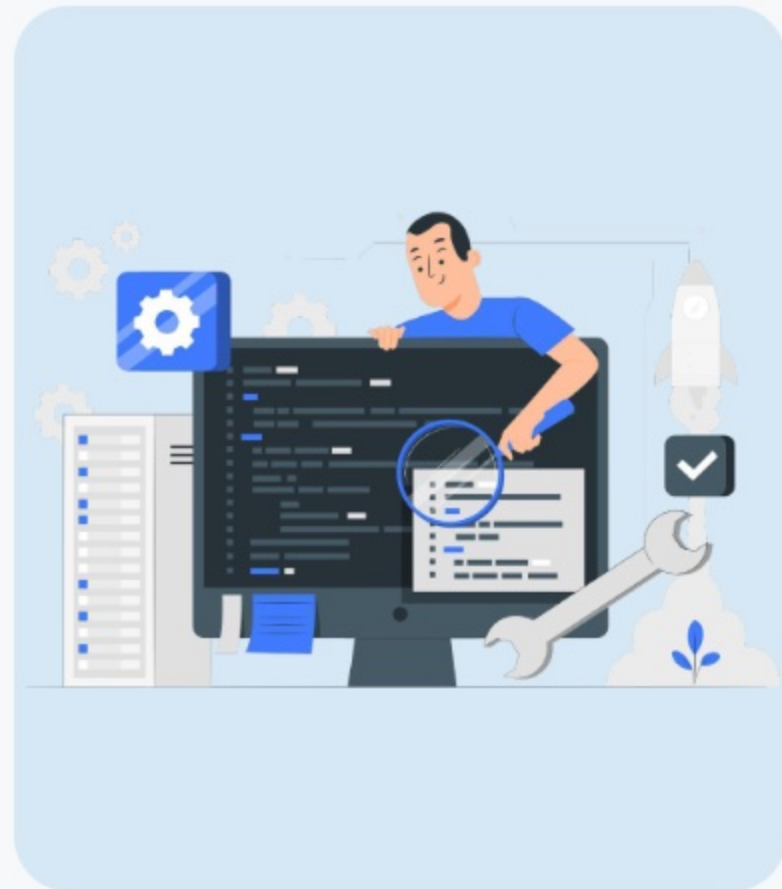
Merge conflicts occur when two developers have **changed the same lines in a file** or if a developer **deleted a file while another developer was modifying it**.

❖ CONFLICT

A merge conflict **only affects the developer** who conducts the merge. The file is marked as **conflicted** and the **merge process is halted**. The developer must then resolve the conflict.

❖ RESOLUTION

There are various **command-line tools available in Git** to resolve merge conflicts. Many third-party tools support **streamlined merge conflict support features**.



Business Scenario

Table of Contents



Business Scenario

Read the business scenario below and determine the most appropriate solutions for the company's requirements.



Cosmic Fashion House, renowned for its exquisite range of high-end fashion clothing from casual to formal attire, boasts a global presence with stores in the United States, Italy, France, and Australia. Leveraging Salesforce, the company expertly manages CRM data and streamlines a multitude of internal operations, including HR management and product planning. Its Salesforce development teams are strategically dispersed across various global locations.

Presently, the company is navigating the complexities of enhancing its development process to efficiently integrate new features, address bugs, and roll out updates for a diverse suite of applications. This endeavor calls for the expertise of a Salesforce architect to fully harness the capabilities of a version control system, employing optimal branching, versioning, and merging strategies to meet the dynamic demands of the company.

Requirements & Solutions

[Table of Contents](#) 

Requirement 1



SCENARIO

The company has multiple development teams that work on separate streams of work, with different timelines and releases. Each team typically works on different code and configurations. The company is exploring the use of a source control system to track code and configuration changes and minimize the risk of unwanted changes. The architect needs to outline the benefits of using a source control system for the company's requirement. Also, a suitable branching strategy is needed to manage the complexity of releases efficiently.

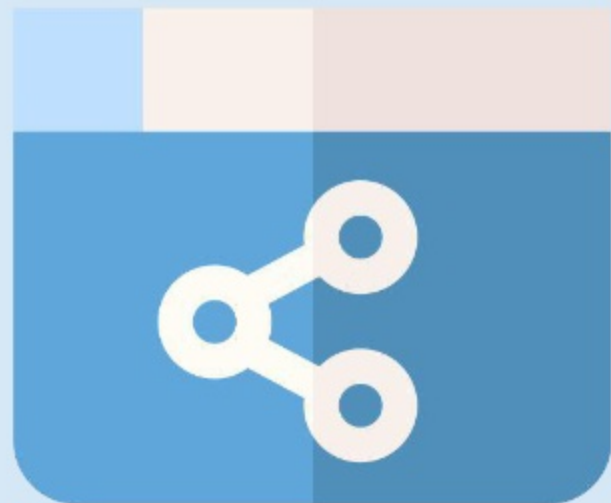


Solution 1



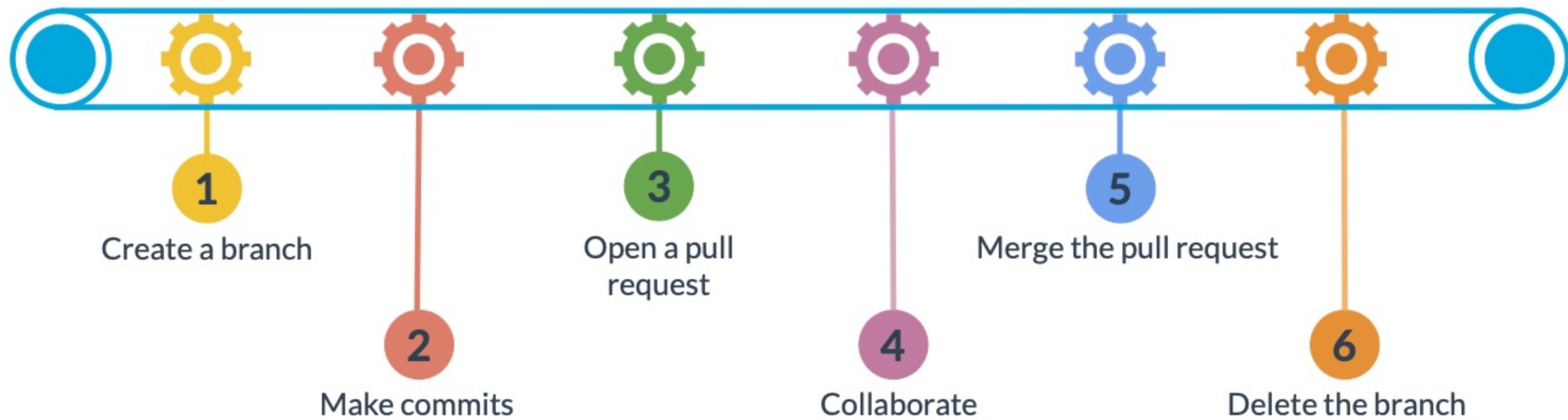
SOLUTION

A source control system like Git is suitable for the company's requirement since it allows **distributed teams to work in isolation**. It would also allow the teams to **back up code and configuration changes**, enabling **rollbacks** in case of unwanted changes. The **GitHub Flow** can be used as the branching strategy for the source control workflow. It allows the use of **multiple feature branches** which can be deployed independently. This suits the company's need for handling multiple release timelines. The development teams can work on feature branches and merge them into the main branch when they are ready for deployment.



Solution 1

GitHub Flow

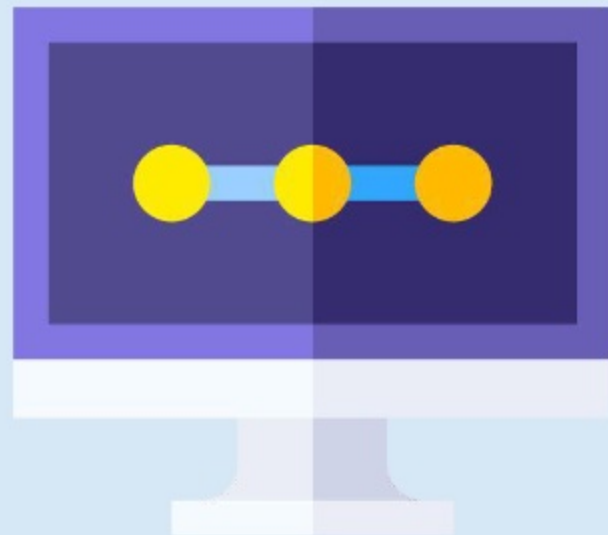


Requirement 2



SCENARIO

Following the architect's recommendation, the company has started using Git as the source control system. Salesforce Extensions for VS Code is used with Salesforce CLI to update various sandboxes from source control. A development team in Australia uses multiple sandboxes and is considering the use of Gitflow as its branching strategy. The strategy emphasizes the use of two main branches called Master and Develop. There are also three supporting branches called Feature, Release, and Hotfix. The architect needs to recommend a suitable approach for development using this branching strategy.



Solution 2



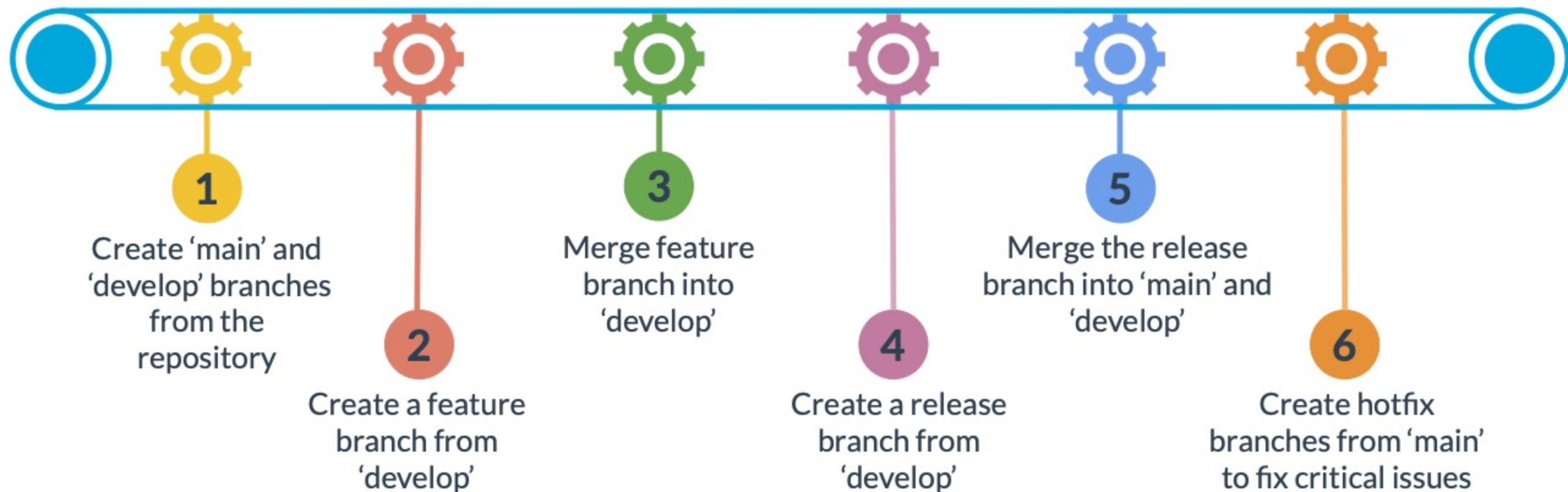
SOLUTION

When using the **Gitflow** workflow, each developer should work off of a **Feature** branch, which is pulled from the **Develop** branch, and then the Feature branch should **merged back into the Develop branch**. This allows for **continuous integration and testing** in the **Develop** branch, which aligns with agile development practices. It is important to note that using feature branches would allow **tracking each feature or change separately** and **maintain a history of changes** made by each developer. However, **code reviews** should be performed before merging branches.



Solution 2

Gitflow Workflow

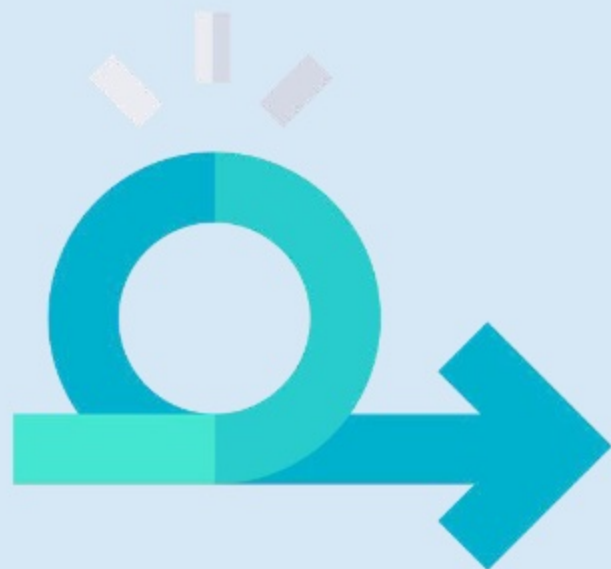


Requirement 3



SCENARIO

A development team working on an Experience Cloud project has completed a sprint and would like to deploy the changes made in the sprint after business approval. However, the team will also immediately begin the next sprint. The architect needs to recommend a suitable strategy to handle the deployment. In addition, two developers are currently working on the Case object and the same Apex classes that reference the Case object. One of them has finished their work early and merged the changes into the Develop branch. The architect needs to provide any related considerations.

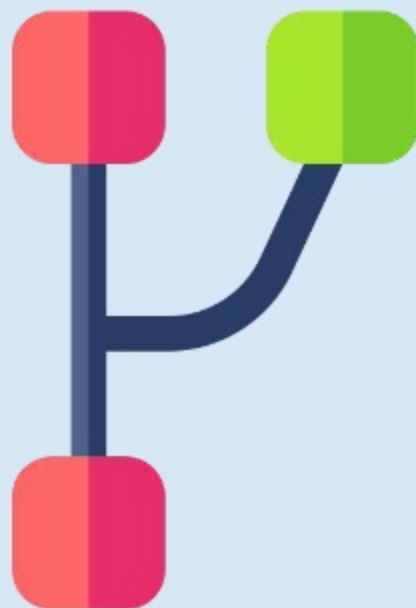


Solution 3 (Part 1)



SOLUTION

For the given requirement, a **Release branch** can be created from the **Develop branch** for the completed sprint. After business approval, the changes made in the sprint can be deployed from the Release branch. For the new sprint, development can be started in **new Feature branches**. After the completion of each feature, its **Feature branch** can be **merged into the Develop branch** for integration and testing. This would allow continuous development and staging of features for release.

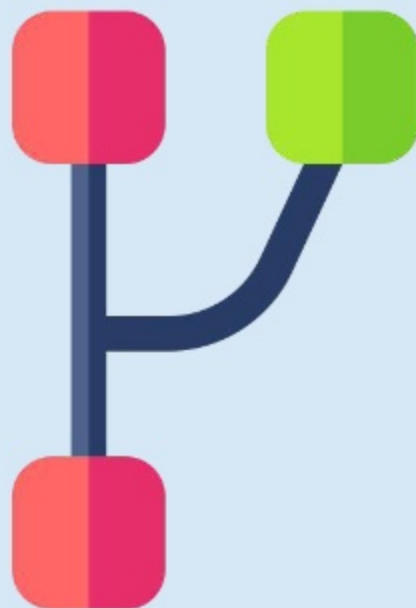


Solution 3 (Part 2)



SOLUTION

The second developer working on the Case object and the same Apex classes could potentially see a **merge conflict** when they create a **pull request** for merging changes in their Feature branch into the Develop branch. They would need to **resolve the conflict** using Git commands to merge the two branches successfully.



Requirement 4



SCENARIO

A development team in the United States is working on the next development phase of a Sales Cloud customization project that requires a lot of development work. The project has 15 Apex developers building new functionality and fixing bugs. The architect needs to recommend a suitable branching strategy that mitigates the risk of developers overwriting each others' changes. Also, the project manager is concerned about critical production issues occurring in the middle of development. A suitable strategy is required to address the issues.

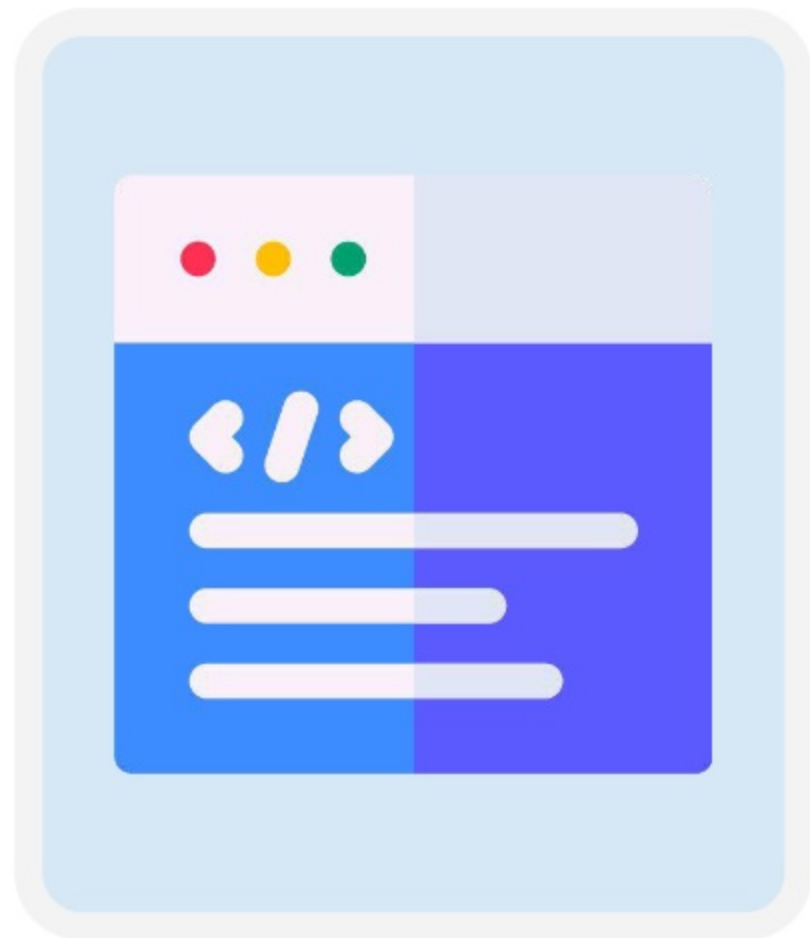


Solution 4



SOLUTION

For the branching strategy, developers should **build new functionality in new branches** and **merge their changes into a common branch for testing**. This would **isolate each developer's work** until it's ready for integration, reducing the risk of conflicts. To address critical production issues immediately without affecting the ongoing development work, **separate branches** should be created for **current development and production bug fixes**. Once the production fix is tested and ready, it can be **merged and deployed** without waiting for the current development cycle to complete.



Learn More

- [!\[\]\(6302aad5aed157b291fddf37b4870784_img.jpg\) Plan for Changes to Your Org](#)
- [!\[\]\(a9ca2c237943a6d0a9f22252f295b6f3_img.jpg\) Git and GitHub Basics](#)
- [!\[\]\(9a01a64e0b4ff865df7d32ee7991fe8b_img.jpg\) Track Changes Between Your Project and Org](#)
- [!\[\]\(6aefe9a3d997eb8b55c40ecd5fa7053f_img.jpg\) Comparing Git workflows](#)
- [!\[\]\(baa8f8ba8c970db55300f5bb45bb3460_img.jpg\) Design Patterns for Salesforce Git Branching Strategies](#)
- [!\[\]\(a6e28495607b2299466d3d5d3193848c_img.jpg\) Semantic Versioning 2.0.0](#)

